Create multisession masks

Ian Durbach & Cornelia Oedekoven

Covered in this tutorial

- Create one trap-buffer polygon for each session
- Crop the elevation-constrained mask to each session
- Create a matching `userdist` matrix for each session mask
- Create a combined multisession survey mask for summaries
- Save multisession mask objects for model fitting

Introduction

Single-session SECR models use one mask. Multisession models use a list of masks, with one mask for each session. This matters in Namkha because the active cameras and camera effort differ between the two temporal blocks created in the capture-history tutorial.

The spatial covariates and elevation constraints were already created in the previous tutorial. This script is mainly about subsetting those objects so that the multisession model gets masks and custom distance matrices that match each session.

Preparations

The multisession mask script loads both the mask file and the capture-history file. The mask file provides `mask_elev` and `userd_elev`; the capture-history file provides `traps_ms`, the list of session-specific traps objects.

```
# Packages needed
library(secr)
library(sf)

mask_file <- "tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_mask.RData"
```

```
capthist_file <- "tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_capthist.RData
output_file <- "tutorials/namkha_advanced/output/namkha_advanced_secr_inputs_msmask.RData"

# Load the mask and capthist objects created in scripts 01 and 02
load(mask_file)
load(capthist_file)
```

Create a trap-buffer polygon for each session

Each session mask is defined by buffering the traps active in that session. The buffer distance is the same as in the single-session fitting mask, but the crop is session-specific.

```
# ADVANCED ISSUE: in a multisession analysis we need one mask for each session.
# Here we create a polygon around the traps in each session, and then use those
# polygons below to crop the full elevation-constrained mask.
# Each session's mask should be within mask_buffer of the traps for that session.
mask_per_session_sf <- list()
for (i in seq_along(traps_ms)) {
  mask_per_session_sf[[i]] <- st_as_sf(traps_ms[[i]], coords = c("x", "y"), crs = my_crs) %>%
    st_buffer(mask_buffer) %>%
    st_union()
}
names(mask_per_session_sf) <- names(traps_ms)
```

Crop the elevation-constrained mask for each session

The starting point is `mask_elev`, not the unconstrained full mask. This keeps the same elevation rule used in the full and shortened analyses.

```
# Convert the full elevation-constrained mask to an sf geometry object so we can
# test which mask points fall inside each session polygon.
mask_elev_sf <- st_as_sf(mask_elev, coords = c("x", "y"), crs = my_crs) %>%
  st_geometry()

# Create the list of masks used in the multisession analysis.
mask_ms <- list()
for (i in seq_along(mask_per_session_sf)) {
  # binary indicator of whether mask points are in the session mask polygon
  mask_in <- lengths(st_intersects(mask_elev_sf, mask_per_session_sf[[i]])) > 0
```

```
  mask_ms[[i]] <- subset(mask_elev, subset = mask_in)
}
names(mask_ms) <- names(traps_ms)
```

The object `mask_ms` is a named list. The names must match the sessions in `ch_ms`, otherwise `secr.fit()` cannot reliably pair each session with its mask.

Create matching user-distance matrices

Because the advanced models use the gorge-avoiding `userdist` object, each session mask also needs its own matching distance matrix. The columns are subset with the same logical indicator used for the mask.

```
# This block is only needed if the analysis uses a custom user-distance matrix,
# for example because there is a movement barrier such as the gorge in Namkha.

# Use the same session-specific mask indicator as above to subset the columns
# of the full userdist matrix.
userd_ms <- list()
for (i in seq_along(mask_per_session_sf)) {
  mask_in <- lengths(st_intersects(mask_elev_sf, mask_per_session_sf[[i]])) > 0
  userd_ms[[i]] <- userd_elev[, mask_in, drop = FALSE]
}
names(userd_ms) <- names(traps_ms)
```

Create masks for combined multisession summaries

The session masks are needed for model fitting. A combined mask is useful for reporting over the spatial footprint of the multisession study.

```
# It is also useful to have a single mask covering the union of all session
# masks, for example when summarising results across the full multisession study.

# Create a mask polygon that is the union of all session masks
all_sess_masks <- mask_per_session_sf[[1]]
for (i in 2:length(mask_per_session_sf)) {
  all_sess_masks <- st_union(all_sess_masks, mask_per_session_sf[[i]])
}
```

```r
# Crop the survey-area elevation mask to the union of all session masks.
# This gives a single survey-area mask covering all sessions combined.
mask_survey_area_elev_sf <- st_as_sf(mask_survey_area_elev, coords = c("x", "y"), crs = my_cr
inside_all_sessions <- lengths(st_intersects(mask_survey_area_elev_sf, all_sess_masks)) > 0
mask_all_sess_survey <- subset(mask_survey_area_elev, subset = inside_all_sessions)

# If using barriers/non-Euclidean movement distances, make the matching userdist
# object for the combined multisession survey mask (optional)
userd_all_sess_survey <- userd_survey_area_elev[, inside_all_sessions, drop = FALSE]
```

The later results script reports over `mask_survey_area_elev`, which keeps the spatial scale consistent with the full and shortened analyses. `mask_all_sess_survey` remains useful when the teaching aim is to summarise the spatial footprint covered by all sessions combined.

Save multisession mask objects

The saved file contains both the session-specific fitting objects and the combined multisession summary objects.

```r
# Save all multisession mask objects needed later in the workflow
save(
  mask_ms, mask_all_sess_survey,
  mask_per_session_sf, all_sess_masks,
  userd_ms, userd_all_sess_survey,
  file = output_file
)
```

What comes next

The next tutorial fits four related advanced model sets: full survey, shortened survey, multi-session, and sex-specific full-survey models.